

Church Account System

Product Requirements Document

Church Account System – Product Requirements Document

1. Overview

Product Name: Church Account

Purpose: A dual-platform (web + mobile) accounting system for recording and tracking Sunday church collections and expenditures. Designed for two non-technical users (father and son) who may enter data independently from different devices, sometimes offline.

2. Users

User	Device	Usage Pattern
Father (50+)	Desktop browser	Primary data entry every Sunday
Son	Android/iOS phone	Secondary entry; sometimes inputs on father's behalf

Both users share a single database. There is no login or auth – the app is private by URL/device.

3. Tech Stack

Layer	Technology
Web frontend	Next.js 15 (App Router), Tailwind CSS
Web backend	Next.js API Routes (serverless)
Database	Turso (LibSQL / SQLite at the edge)
DB client	@libsql/client (async, pre-built binaries)
Excel export	SheetJS (xlsx)
Mobile	Expo SDK 52, React Native 0.76, Expo Router v4
Offline storage	@react-native-async-storage/async-storage
Network detection	@react-native-community/netinfo
Deployment	Vercel (web), EAS or local Gradle (mobile APK)

4. Repository Layout

```
church-account/  
+-- src/                                # Next.js web app  
|   +-- app/  
|       +-- layout.tsx                  # Root layout with sidebar nav  
|       +-- page.tsx                    # Dashboard  
|       +-- history/  
|           +-- page.tsx                # Income records list  
|       +-- expenditure/  
|           +-- page.tsx                # New expenditure form  
|           +-- history/  
|               +-- page.tsx           # Expenditure records list  
|       +-- api/  
|           +-- entries/  
|               +-- route.ts           # GET list, POST create  
|               +-- [id]/  
|                   +-- route.ts       # GET one, PUT update, DELETE  
|           +-- expenditure/  
|               +-- route.ts  
|               +-- [id]/  
|                   +-- route.ts  
|           +-- export/  
|               +-- route.ts           # Excel export for income  
|               +-- expenditure/  
|                   +-- route.ts       # Excel export for expenditure  
|   +-- components/  
|       +-- EntryForm.tsx              # Income entry form  
|       +-- ExpenditureForm.tsx        # Expenditure entry form  
|   +-- lib/  
|       +-- db.ts                      # LibSQL client + table init  
+-- mobile/  
|   +-- app/  
|       +-- _layout.tsx                # Root stack + OnlineContext provider  
|       +-- (tabs)/  
|           +-- _layout.tsx            # Bottom tab navigator  
|           +-- index.tsx              # Dashboard  
|           +-- income.tsx             # New income form  
|           +-- history.tsx            # Income records  
|           +-- expenditure.tsx        # New expenditure form  
|           +-- exp-history.tsx        # Expenditure records  
|       +-- edit-income/  
|           +-- [id].tsx               # Edit income entry  
|       +-- edit-exp/  
|           +-- [id].tsx               # Edit expenditure entry  
|   +-- components/  
|       +-- AmountRow.tsx              # Row with church + project inputs  
|       +-- OfflineBanner.tsx          # Connectivity status strip
```

```

|   +--- constants/
|   |   +--- api.ts           # apiFetch, fc, isNetworkError
|   |   +--- categories.ts    # Category and service type lists
|   |   +--- colors.ts        # Shared colour palette
|   +--- hooks/
|   |   +--- useOfflineSync.ts # NetInfo listener, flush queue, React Context
|   +--- utils/
|       +--- storage.ts        # AsyncStorage cache + write queue helpers
+--- package.json              # Root (web)

```

5. Data Model

Table: entries (income)

Column	Type	Notes
id	INTEGER PK AUTOINCREMENT	
date	TEXT	YYYY-MM-DD
service_type	TEXT	One of 7 service types
{category}_church	REAL	One column per category per side
{category}_project	REAL	
total_church	REAL	Sum of all _church columns
total_project	REAL	Sum of all _project columns
grand_total	REAL	total_church + total_project

Table: expenditures

Identical structure to **entries** but uses the 14 expenditure categories.

Income Categories (14)

Offering, Tithe, Sunday School, Covenant Offering, Thanksgiving, Holy Communion, Special Thanksgiving/Gift, Fellowship, Dedication, Sow A Seed, Pastor's Appreciation, Harvest, Project Support, LCC

Expenditure Categories (14)

Transportation, Premise, 25%, Gift, Battery, Fuel, Electricity, LCC/DCC, Entertainment, Pastor's Appreciation, Stationeries, Accessories, PHCN, Assessment

Service Types (7)

First Service, Second Service, Third Service, Combined Service, Special Service, Youth Service, Children Service

6. Web App Features

6.1 Dashboard (/)

- Summary cards: Total Income, Total Expenditure, Net Balance, This Month figures
- Entry counts per type
- Recent income and expenditure tables (last 5 each)
- Quick-action buttons to each form and history page
- `force-dynamic` – never cached at build time

6.2 New Income Form (/)

- Date input (YYYY-MM-DD, defaults to today)
- Service Type dropdown
- 14-row table: Category / Church amount / Project amount
- Sticky frosted-glass footer with live Church total, Project total, Grand Total, and Submit button
- Posts to `POST /api/entries`

6.3 Income History (/history)

- Full list, newest first
- Each row shows date, service type, grand total
- Expandable row: per-category breakdown + church/project sub-totals
- Edit button ? inline edit form (PUT)
- Delete button with confirmation (DELETE)
- Export to Excel button ? `GET /api/export`
- `force-dynamic`

6.4 New Expenditure Form (/expenditure)

- Same layout as income form
- Posts to `POST /api/expenditure`

6.5 Expenditure History (/expenditure/history)

- Same as income history
- Export ? `GET /api/export/expenditure`

6.6 Excel Export Format

Both exports produce a `.xlsx` file matching the original spreadsheet format:

- Row 1: merged header “CHURCH ACCOUNT – [Year]”
- Row 2: column headers – Date, Service Type, then per category: CHURCH and PROJECT sub-headers
- One data row per entry
- Totals row at the bottom

6.7 Navigation / Layout

- Desktop: fixed left sidebar (256 px), grouped Income / Expenditure sections

- Mobile: horizontal scrollable top nav bar
 - Indigo/violet gradient logo with + symbol
 - Tailwind design: slate card backgrounds, indigo / emerald / rose accents
-

7. Mobile App Features

7.1 Bottom Tab Navigator

Five tabs: Dashboard ? New Income ? Income Records ? New Exp. ? Exp. Records

7.2 Dashboard Tab

- Income and expenditure stat cards (total, count, this-month)
- Net balance card (green if positive, orange if negative)
- Quick action buttons (4-up grid)
- Recent income list (last 5) with (pending) pending indicator
- Recent expenditure list (last 5)
- Pull-to-refresh fetches from API

7.3 New Income / New Expenditure Tabs

- Date input + Service Type picker (custom dropdown)
- AmountRow component for each category (church + project numeric inputs)
- Sticky footer: Church total, Project total, Grand Total, Save button
- On save: optimistic write to cache ? try API ? queue if offline

7.4 Income / Expenditure Records Tabs

- Cache-first load (instant on repeat opens)
- Background API refresh when online
- Pull-to-refresh forces API fetch
- Expandable cards with category chips, sub-totals, Edit / Delete actions
- Pending entries show (pending) badge and dashed border
- Delete works offline (queued for later sync)

7.5 Edit Screens (stack, not tabs)

- Loads entry from local cache first (no network required)
 - Same form layout as new entry screens
 - Shows “(offline) Offline” note when disconnected
 - On save: optimistic cache update ? try API PUT ? queue if offline
-

8. Offline Architecture

Device

AsyncStorage

Server (Vercel / Turso)

```
INCOME_CACHE  ?---- sync -----> GET /api/entries
EXP_CACHE     ?---- sync -----> GET /api/expenditure
WRITE_QUEUE   -----> POST / PUT / DELETE
```

useOfflineSync hook (mounted in _layout.tsx):

```
NetInfo.addListener
  ? isOnline flips true
  ? flushQueue() processes ops in order
  ? after flush: re-fetch both caches from server
  ? expose { isOnline, isSyncing, pendingCount }
    via OnlineContext to all screens
```

Write Queue Entry Shape

```
interface QueuedOp {
  opId: string;           // Date.now() string -- unique per op
  type: "POST" | "PUT" | "DELETE";
  path: string;           // e.g. "/api/entries/5"
  body?: Record<string, unknown>;
  tempId?: number;       // negative number used for offline POSTs
}
```

Temp ID Pattern

- New entries created offline get `id = -Date.now()` (always negative, never collides with server auto-increment)
- Stored in cache with `pending: true`
- When POST flushes: server response carries real `id`; cache entry and any subsequent queue ops referencing `tempId` are updated in place

OfflineBanner States

State	Colour	Message
Offline, no pending	Orange	(offline) You're offline – showing saved data
Offline + pending ops	Orange	(offline) Offline – N entries will sync when connected
Syncing	Indigo	(syncing) Syncing N pending entries?
Online + pending ops	Amber	(pending) N entries pending sync
Online, clean	hidden	–

9. API Routes

Method	Path	Description
GET	/api/entries	All income entries, newest first
POST	/api/entries	Create income entry
GET	/api/entries/:id	Single income entry
PUT	/api/entries/:id	Update income entry
DELETE	/api/entries/:id	Delete income entry
GET	/api/expenditure	All expenditure entries
POST	/api/expenditure	Create expenditure entry
GET	/api/expenditure/:id	Single expenditure entry
PUT	/api/expenditure/:id	Update expenditure entry
DELETE	/api/expenditure/:id	Delete expenditure entry
GET	/api/export	Download income Excel file
GET	/api/export/expenditure	Download expenditure Excel file

10. Environment Variables

Variable	Where	Value
TURSO_DATABASE_URL	Vercel + local <code>.env.local</code>	<code>libsql://your-db.turso.io</code>
TURSO_AUTH_TOKEN	Vercel + local <code>.env.local</code>	Turso auth token
EXPO_PUBLIC_API_URL	mobile/.env	<code>https://your-app.vercel.app</code>

11. Deployment

Web (Vercel)

1. Push to GitHub
2. Import repo in Vercel dashboard
3. Set `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN` in Vercel environment variables
4. Deploy – database tables are auto-created on first request via `db.batch()` DDL

Mobile APK via EAS Build (recommended)

```
cd mobile
npm install --legacy-peer-deps
npm install -g eas-cli
eas login
eas build --platform android --profile preview
# Expo provides a direct APK download link (~10-15 min)
```

Mobile APK via Local Build

```
cd mobile
npm install --legacy-peer-deps
```

```

npx expo prebuild --platform android
cd android
./gradlew assembleRelease
# Output: android/app/build/outputs/apk/release/app-release.apk
# First build takes ~35-50 min; subsequent builds ~5-8 min

```

12. Key Design Decisions

Decision	Rationale
Turso (LibSQL) over <code>better-sqlite3</code>	Vercel Node 24 cannot compile <code>better-sqlite3</code> C++ bindings; Turso ships pre-built binaries and works in serverless functions
AsyncStorage over SQLite for mobile	Data volume is tiny (~200 entries/year); AsyncStorage is simpler and requires no native compilation step
Negative temp IDs	Simple way to distinguish unsynced entries without a separate pending table; never collides with server auto-increment IDs which always start at 1
Single shared database	Both users sync through the same Turso DB via the Vercel API; no per-user data segregation needed
No authentication	App is private by obscurity (URL not shared publicly); auth would add friction for the 50+ year old primary user
<code>force-dynamic</code> on all DB pages	Prevents Next.js from running DB queries at Vercel build time before environment variables are available